

Preparación de Entrevista — Arquitecto de Software

Antonio García Márquez · Proyecto de referencia: **mini-uber / driver-service**

1. Perfil profesional

Ingeniero de software con experiencia en el diseño y desarrollo de sistemas distribuidos basados en microservicios. Enfoque en arquitecturas escalables, resilientes y mantenibles, con especial atención a las buenas prácticas de diseño (SOLID, DDD, arquitectura hexagonal) y a la calidad del código (pruebas automatizadas, integración continua).

Puntos clave a comunicar

- Capacidad para diseñar sistemas desacoplados y orientados a eventos.
- Experiencia práctica llevando servicios desde el diseño hasta producción.
- Visión de arquitecto: equilibrio entre simplicidad, escalabilidad y coste de mantenimiento.
- Buenas prácticas de documentación y comunicación técnica con el equipo.

2. Arquitectura del proyecto de referencia: mini-uber

mini-uber es un sistema tipo "ride-hailing" (similar a Uber) compuesto por microservicios independientes que colaboran entre sí. **driver-service** es el microservicio responsable de la gestión de los conductores (alta, disponibilidad, ubicación, estado del vehículo, etc.) dentro de dicho ecosistema.

Responsabilidades de driver-service

- Registro y gestión del ciclo de vida de los conductores.
- Actualización y consulta del estado/disponibilidad en tiempo real.
- Exposición de una API para que otros servicios (por ejemplo, el servicio de viajes o de asignación) puedan consultar y reservar conductores disponibles.
- Publicación de eventos de dominio (ej. "conductor disponible", "conductor asignado") para mantener desacoplados a los servicios consumidores.

Decisiones arquitectónicas a defender

Aspecto	Decisión	Justificación
Estilo de arquitectura	Microservicios	Escalado independiente, despliegues autónomos y aislamiento de fallos por dominio.
Comunicación	Síncrona (REST/gRPC) para consultas, asíncrona (eventos) para notificaciones	Reduce el acoplamiento temporal entre servicios y mejora la resiliencia ante caídas puntuales.

Persistencia	Base de datos propia por servicio (database per service)	Evita acoplamiento a nivel de esquema y permite evolución independiente del modelo de datos.
Resiliencia	Circuit breaker / timeouts / reintentos en llamadas entre servicios	Evita el efecto cascada de fallos en un sistema distribuido.

3. Puntos fuertes a destacar

- **Arquitectura** Diseño de sistemas distribuidos desacoplados mediante eventos y APIs bien definidas (contract-first).
- **Calidad** Cultura de testing (unitario, integración, contract testing entre servicios) e integración continua.
- **Escalabilidad** Diseño pensado para el crecimiento horizontal: servicios sin estado, cacheo, particionado de datos.
- **Observabilidad** Logging estructurado, métricas y trazabilidad distribuida para depurar problemas entre microservicios.
- **Liderazgo técnico** Capacidad de guiar decisiones de arquitectura y mentorizar a otros desarrolladores.
- **Comunicación** Traducir decisiones técnicas complejas en términos comprensibles para perfiles no técnicos.

4. Preguntas y respuestas típicas de entrevista

4.1 Preguntas técnicas de arquitectura

¿Por qué elegirías una arquitectura de microservicios frente a un monolito?

Los microservicios permiten escalar y desplegar cada componente de forma independiente, aíslan fallos por dominio y facilitan que equipos distintos trabajen en paralelo. A cambio, aumentan la complejidad operativa (observabilidad, consistencia de datos, comunicación entre servicios), por lo que solo se justifican cuando el tamaño del equipo y del producto lo requiere. Un monolito bien modularizado puede ser preferible en etapas tempranas.

¿Cómo garantizas la consistencia de datos entre microservicios como driver-service y el servicio de viajes?

Evito transacciones distribuidas síncronas (2PC) y opto por consistencia eventual mediante el patrón Saga, coordinando el flujo con eventos de dominio (por ejemplo, "DriverAssigned", "TripCancelled") y compensaciones si un paso falla. Esto prioriza la disponibilidad del sistema sobre la consistencia inmediata.

¿Cómo evitarías que un fallo en un servicio downstream tumbe todo el sistema?

Aplicando patrones de resiliencia: timeouts agresivos, circuit breaker (ej. Resilience4j), reintentos con backoff exponencial, bulkheads para aislar recursos y respuestas degradadas (fallback) cuando sea posible, en lugar de propagar el fallo en cascada.

¿Cómo diseñarías la comunicación entre driver-service y el resto de servicios?

Para consultas puntuales de estado (¿está disponible este conductor?) usaría comunicación síncrona vía REST o gRPC con contratos versionados. Para notificar cambios de estado a múltiples consumidores (asignación, facturación, notificaciones push) usaría eventos asíncronos a través de un bus de mensajes (Kafka/RabbitMQ), desacoplando productores y consumidores en el tiempo.

¿Cómo gestionarías el versionado de la API de driver-service?

Versionado explícito (por ejemplo, en la URL o en cabeceras), manteniendo compatibilidad hacia atrás siempre que sea posible, evolucionando los contratos de forma aditiva y comunicando de forma clara las fechas de deprecación de versiones antiguas.

¿Qué estrategia seguirías para la base de datos de un servicio con alta carga de lectura, como la consulta de conductores disponibles?

Separar lecturas y escrituras (CQRS ligero), añadir una capa de caché (Redis) para las consultas más frecuentes, indexar adecuadamente por ubicación/estado y considerar particionado geográfico si el volumen lo requiere.

4.2 Escalabilidad y rendimiento

¿Cómo escalarías driver-service si el número de conductores activos se multiplicase por 100?

Primero mediría dónde está el cuello de botella (CPU, memoria, I/O de base de datos, red) con pruebas de carga. Después escalaría horizontalmente las instancias sin estado detrás de un balanceador, añadiría caché para las consultas de disponibilidad más frecuentes, optimizaría índices y consideraría particionar los datos por región geográfica si la consulta principal es "conductores disponibles cerca de X".

¿Qué diferencia hay entre escalado horizontal y vertical? ¿Cuándo usarías cada uno?

El escalado vertical añade más recursos (CPU/RAM) a una misma instancia; es simple pero tiene un límite físico y un único punto de fallo. El horizontal añade más instancias y reparte la carga; requiere que el servicio sea sin estado (o externalizar el estado) pero escala prácticamente sin límite y mejora la disponibilidad. En microservicios casi siempre se prioriza el escalado horizontal.

¿Cómo diseñarías el sistema de asignación de conductores para que sea eficiente a nivel geográfico (buscar el conductor más cercano)?

Usaría estructuras de datos e índices geoespaciales (geohash, quad-tree, o índices nativos como PostGIS/Redis GEO) en lugar de calcular distancias contra toda la tabla. Esto permite consultas de "conductores en un radio de X km" en tiempo casi constante en lugar de lineal con el número total de conductores.

¿Qué es el "hot partition" o punto caliente en un sistema distribuido y cómo lo evitarías?

Ocurre cuando una partición o clave concentra desproporcionadamente el tráfico (por ejemplo, todos los conductores de una ciudad muy grande). Se mitiga con un particionado más granular (por zonas más pequeñas), añadiendo un sufijo aleatorio a la clave (salting) o replicando/cacheando esa partición concreta.

¿Cómo medirías y mejorarías la latencia percibida por el usuario final al solicitar un conductor?

Instrumentaría el sistema con trazas distribuidas (OpenTelemetry) para ver el tiempo consumido en cada salto (API gateway, driver-service, matching, base de datos). Con esos datos priorizaría optimizar el camino crítico: cachear datos que no cambian con frecuencia, paralelizar llamadas independientes, y mover trabajo no esencial (notificaciones, analítica) fuera del camino síncrono mediante eventos.

4.3 Seguridad

¿Cómo asegurarías la comunicación y autenticación entre microservicios?

Usaría mTLS o un service mesh (Istio/Linkerd) para cifrar y autenticar el tráfico interno, tokens JWT de corta duración para la identidad del servicio/usuario, y un API gateway que centralice la autenticación de las peticiones externas antes de llegar a los servicios internos.

¿Cómo protegerías datos sensibles de los conductores (documentos, ubicación en tiempo real, datos bancarios)?

Cifrado en tránsito (TLS) y en reposo, minimización de datos (solo almacenar lo estrictamente necesario), tokenización o enmascaramiento de datos sensibles en logs, control de acceso basado en roles (RBAC) y auditoría de accesos. Para datos de pago, delegaría en un proveedor certificado PCI-DSS en lugar de almacenar tarjetas directamente.

¿Cómo evitarías que un servicio comprometido pueda acceder a todos los demás servicios del sistema?

Aplicando el principio de mínimo privilegio: cada servicio solo tiene permisos sobre los recursos que necesita, segmentación de red (network policies), autenticación mutua entre servicios y rotación periódica de credenciales/secrets gestionados con un vault (HashiCorp Vault, AWS Secrets Manager).

¿Qué es OWASP Top 10 y cómo lo tendrías en cuenta al diseñar una API?

Es una lista de los riesgos de seguridad más críticos en aplicaciones web (inyección, autenticación rota, exposición de datos sensibles, control de acceso roto, etc.). En el diseño de la API validaría y sanearía todas las entradas, usaría autenticación/autorización robustas, evitaría exponer información sensible en errores, y aplicaría rate limiting para prevenir abuso.

4.4 Patrones de diseño y DDD

¿Cómo aplicarías Domain-Driven Design (DDD) para definir los límites de driver-service frente a otros servicios (trips, billing, notifications)?

Identificando los "bounded contexts": driver-service es dueño del ciclo de vida y estado del conductor; trips-service es dueño del ciclo de vida del viaje. La comunicación entre contextos se hace mediante un "context mapping" explícito (eventos o APIs), evitando que un servicio manipule directamente el modelo de datos de otro.

¿Qué es un agregado (aggregate) en DDD y cómo lo aplicarías al modelo de conductor?

Un agregado es un conjunto de entidades y value objects que se tratan como una unidad de consistencia transaccional, con una raíz (aggregate root) que controla el acceso. En driver-service, "Driver" sería la raíz del agregado, agrupando su perfil, estado de disponibilidad y ubicación actual, garantizando que cualquier cambio mantenga las invariantes de negocio (por ejemplo, no se puede asignar un viaje a un conductor no disponible).

¿Qué patrones usarías para desacoplar la lógica de negocio de los detalles de infraestructura (framework, base de datos)?

Arquitectura hexagonal (puertos y adaptadores) o Clean Architecture: el dominio define interfaces (puertos) y la infraestructura provee implementaciones (adaptadores). Esto permite testear la lógica de negocio sin depender de una base de datos real y facilita cambiar de tecnología sin tocar el núcleo del dominio.

¿Cuándo usarías CQRS y cuándo sería excesivo aplicarlo?

CQRS (separar modelos de lectura y escritura) tiene sentido cuando los patrones de lectura y escritura son muy distintos (por ejemplo, muchas más lecturas de "conductores disponibles" que escrituras de estado) o cuando se necesita escalar cada lado de forma independiente. Es excesivo para dominios simples con CRUD básico, donde añadiría complejidad sin un beneficio claro.

4.5 Testing y calidad

¿Qué estrategia de testing aplicarías en un ecosistema de microservicios?

La pirámide de testing: muchas pruebas unitarias rápidas para la lógica de dominio, pruebas de integración para verificar la interacción con la base de datos o colas de mensajes, contract testing (ej. Pact) para validar que los contratos entre driver-service y sus consumidores no se rompen, y un número reducido de pruebas end-to-end para los flujos críticos.

¿Qué es el contract testing y por qué es especialmente útil en microservicios?

Permite verificar que un consumidor y un proveedor de una API cumplen un contrato acordado (esquema de petición/respuesta) sin necesidad de desplegar ambos servicios juntos. Evita regresiones cuando un equipo cambia su API sin coordinarse con los consumidores, algo muy común en sistemas con muchos servicios independientes.

¿Cómo probarías la resiliencia de driver-service ante fallos de sus dependencias?

Con pruebas de caos controladas (chaos engineering): simular latencia alta, caídas del servicio de base de datos o de la cola de mensajes, y verificar que los mecanismos de circuit breaker, timeouts y fallback se comportan como se espera, sin degradar el resto del sistema.

4.6 DevOps, CI/CD y despliegue

¿Cómo diseñarías el pipeline de CI/CD para driver-service?

Build y pruebas unitarias/integración en cada commit, análisis estático y de seguridad (linters, SAST), construcción de una imagen inmutable (Docker), despliegue automático a un entorno de staging, pruebas de contrato/smoke tests, y despliegue a producción mediante una estrategia progresiva (canary o blue-green) con posibilidad de rollback automático si las métricas empeoran.

¿Qué diferencia hay entre un despliegue blue-green y uno canary?

Blue-green mantiene dos entornos idénticos (uno activo, uno en espera) y conmuta todo el tráfico de golpe a la nueva versión, permitiendo rollback instantáneo. Canary despliega la nueva versión a un pequeño porcentaje de tráfico real, se observan las métricas y se va incrementando gradualmente, reduciendo el radio de impacto si algo falla.

¿Cómo gestionarías la configuración y los secretos de driver-service en distintos entornos (dev, staging, producción)?

Externalizando la configuración del código (12-factor app), usando variables de entorno o un servicio de configuración centralizado, y gestionando los secretos con una herramienta dedicada (Vault, AWS Secrets Manager) en lugar de ficheros de configuración en el repositorio, con distintos valores/permisos por entorno.

4.7 Bases de datos y almacenamiento

¿Cómo diseñarías el modelo de datos de driver-service para soportar actualizaciones de ubicación muy frecuentes sin degradar el rendimiento?

Separaría el dato de "ubicación en tiempo real" (alta frecuencia de escritura, se puede perder ocasionalmente) del dato "perfil del conductor" (baja frecuencia, debe ser consistente). La ubicación en tiempo real la guardaría en un almacén de baja latencia como Redis en lugar de la base de datos relacional principal, para no saturarla con escrituras constantes.

¿Cuándo elegirías una base de datos SQL frente a una NoSQL para un microservicio?

SQL cuando el dominio tiene relaciones complejas, necesita transacciones ACID fuertes y un esquema bien definido (por ejemplo, el perfil y estado del conductor). NoSQL (documental, clave-valor o series temporales) cuando el volumen de escritura es muy alto, el esquema es flexible o se necesita escalado horizontal nativo (por ejemplo, histórico de ubicaciones o eventos de telemetría).

¿Cómo migrarías el esquema de la base de datos de driver-service sin causar downtime?

Con migraciones "expand-contract": primero se añade la nueva estructura sin eliminar la antigua (expand), se despliega el código que escribe en ambas, se migran los datos existentes, se cambia la lectura a la nueva estructura, y solo cuando todo está validado se elimina la estructura antigua (contract). Esto evita romper compatibilidad durante despliegues graduales.

4.8 Mensajería y arquitectura orientada a eventos

¿Qué garantías de entrega ofrece un sistema de mensajería como Kafka y cómo afectan al diseño de driver-service?

Normalmente "at-least-once": un mensaje puede procesarse más de una vez ante fallos o reintentos. Por eso los consumidores de eventos de driver-service deben ser idempotentes (por ejemplo, usando un identificador único de evento para no aplicar dos veces el mismo cambio de estado).

¿Qué es el patrón Outbox y por qué es relevante al publicar eventos desde driver-service?

Resuelve el problema de actualizar la base de datos y publicar un evento de forma atómica: en lugar de escribir en la base de datos y luego publicar al bus de mensajes (dos operaciones no atómicas que pueden desincronizarse), se escribe el evento en una tabla "outbox" dentro de la misma transacción, y un proceso aparte (o CDC) lo publica de forma fiable al bus de mensajes.

¿Cómo evitarías el acoplamiento excesivo entre servicios al diseñar los eventos de dominio?

Diseñando eventos que expresen "hechos de negocio" (ej. "DriverBecameAvailable") en lugar de replicar la estructura interna de la base de datos, versionando los esquemas de eventos de forma compatible hacia atrás, y evitando que los consumidores dependan de detalles de implementación internos del servicio emisor.

4.9 Observabilidad y monitorización

¿Qué métricas clave monitorizarías en driver-service en producción?

Latencia y tasa de error de la API (p50/p95/p99), tasa de conductores disponibles frente a la demanda, tiempo de procesamiento de eventos, saturación de la base de datos/caché, y métricas de negocio como tiempo medio hasta encontrar un conductor disponible.

¿Cómo depurarías un problema intermitente de latencia que solo aparece en producción y afecta a varios servicios?

Usando trazas distribuidas (OpenTelemetry/Jaeger) para seguir una petición a través de todos los servicios implicados e identificar en qué salto concreto se produce la latencia, correlacionando con métricas de infraestructura (CPU, memoria, I/O) y logs estructurados con un identificador de correlación común.

¿Qué diferencia hay entre logs, métricas y trazas, y cómo se complementan?

Los logs registran eventos discretos con contexto detallado; las métricas son series numéricas agregadas en el tiempo (ideales para alertas y dashboards); las trazas muestran el recorrido completo de una petición a través de múltiples servicios. Juntas forman los "tres pilares de la observabilidad" y permiten pasar de "algo va mal" (métrica/alerta) a "por qué va mal" (traza) a "qué pasó exactamente" (log).

4.10 Cloud e infraestructura

¿Cómo desplegarías driver-service en un entorno cloud con alta disponibilidad?

Contenedores orquestados con Kubernetes, desplegados en al menos dos zonas de disponibilidad, con auto-scaling horizontal basado en métricas de carga, health checks/liveness y readiness probes, y una base de datos gestionada con réplicas para tolerancia a fallos.

¿Qué ventajas y riesgos tiene usar servicios gestionados (managed services) de un proveedor cloud frente a autogestionar la infraestructura?

Ventajas: menor carga operativa, alta disponibilidad y parcheo gestionados por el proveedor, mayor velocidad de entrega. Riesgos: vendor lock-in, menor control sobre configuraciones específicas y coste potencialmente mayor a gran escala. La decisión depende del tamaño del equipo de plataforma y de cuánto valor aporta controlar ese detalle frente a delegarlo.

4.11 Preguntas de comportamiento y experiencia

Cuéntame sobre un momento en el que tuviste que tomar una decisión de arquitectura difícil.

Estructura la respuesta con el método STAR: Situación (contexto del proyecto), Tarea (el problema concreto a resolver), Acción (las alternativas evaluadas y por qué elegiste una) y Resultado (impacto medible: rendimiento, mantenibilidad, reducción de incidencias). Ejemplo: decidir entre acoplar dos servicios vía llamada síncrona o desacoplarlos con eventos, explicando el trade-off de latencia frente a resiliencia.

¿Cómo gestionas el desacuerdo técnico dentro de un equipo?

Fomento la discusión basada en datos y no en opiniones: pruebas de concepto, benchmarks o documentos de diseño (ADR - Architecture Decision Records) que comparen alternativas. Busco el consenso, pero si no se alcanza, se toma una decisión con un responsable claro, documentando el porqué para poder revisarla más adelante si las circunstancias cambian.

Describe una vez que tuviste que aprender una tecnología nueva rápidamente.

Explica tu proceso de aprendizaje (documentación oficial, prototipos pequeños, pair programming) y cómo lo aplicaste de forma segura en el proyecto real, por ejemplo introduciendo la tecnología primero en un componente de bajo riesgo antes de generalizarla.

¿Cómo priorizas entre entregar rápido y mantener la calidad/arquitectura limpia?

Reconozco la deuda técnica como una herramienta válida cuando se gestiona de forma consciente: documento la decisión, su coste futuro y planifico cuándo abordarla. La calidad no es negociable en aspectos críticos (seguridad, integridad de datos), pero sí puede haber margen en aspectos no críticos a corto plazo.

¿Cómo comunicas decisiones de arquitectura a perfiles no técnicos (negocio, producto)?

Traduzco el impacto técnico a términos de negocio: tiempo, coste, riesgo y beneficio. Evito la jerga y uso analogías o diagramas simples, centrándome en qué gana el negocio (fiabilidad, velocidad de entrega futura) en lugar de en el detalle de implementación.

Cuéntame sobre un fallo grave en producción del que fueras responsable. ¿Qué pasó y qué aprendiste?

Usa STAR y sé honesto: describe el incidente, el impacto real (usuarios/tiempo de caída), la causa raíz identificada mediante un post-mortem sin buscar culpables, y las acciones concretas implementadas después (nuevas alertas, tests, cambios de proceso) para que no vuelva a ocurrir. Los entrevistadores valoran la capacidad de aprender y mejorar procesos, no la ausencia de errores.

¿Cómo convences a un equipo o a negocio de invertir tiempo en pagar deuda técnica en lugar de construir nuevas funcionalidades?

Cuantifico el impacto de la deuda en términos que negocio entienda: incidencias recurrentes, tiempo perdido en despliegues, riesgo de un fallo mayor, o ralentización de la velocidad de entrega futura. Propongo abordarla de forma incremental, integrada en el propio desarrollo de features, en lugar de pedir un "parón" completo para refactorizar.

Describe cómo mentorizas o ayudas a crecer a desarrolladores menos senior en tu equipo.

Combino code reviews constructivas (explicando el porqué, no solo el qué), pair/mob programming en tareas complejas, y fomento la autonomía dándoles problemas con margen para proponer su propia solución antes de intervenir. También comparto documentación y ADRs para que entiendan el contexto de las decisiones existentes.

¿Cómo manejas la presión de plazos ajustados sin comprometer la estabilidad del sistema?

Negocio el alcance antes que la calidad: identifico con el equipo de producto qué es imprescindible y qué se puede posponer, comunico riesgos de forma temprana y transparente, y reservo tiempo para pruebas de los flujos críticos aunque se reduzca el de funcionalidades secundarias.

5. Preguntas para hacer al entrevistador

- ¿Cómo está estructurado el equipo de arquitectura y qué grado de autonomía tienen los equipos de producto sobre sus propios microservicios?
- ¿Qué herramientas de observabilidad (logging, métricas, trazas) utilizan actualmente para monitorizar el sistema en producción?
- ¿Cómo gestionan actualmente la consistencia de datos entre servicios? ¿Usan algún patrón como Saga o eventos de dominio?
- ¿Cuál es el mayor reto de arquitectura al que se han enfrentado en el último año?
- ¿Qué proceso siguen para tomar y documentar decisiones de arquitectura (ADRs, RFCs, etc.)?
- ¿Cómo es el proceso de despliegue y cuál es la frecuencia de entrega a producción?